

```
public class Driver {

    public static void main(String[] args) {

        // Q1
        System.out.println(factorial(5));
        // Q2
        int len = 10;
        for (int i = 1; i <= len; i++) {
            System.out.print(fibonacci(i) + " ");
        }

    }

    public static int factorial(int n) {
        if (n <= 1) {
            return 1;
        }
        return factorial(n - 1) * n;
    }

    public static int fibonacci(int n) {
        if (n == 0 || n == 1) {
            return n;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

}
```

```
public class TreeNode {

    int key;
    TreeNode right;
    TreeNode left;

    public TreeNode(int data) {
        this.key = data;
        this.right = null;
        this.left = null;
    }

}
```

```

}
public class BinarySearchTree {

    TreeNode root;

    public void insert(int key) {
        TreeNode newNode = new TreeNode(key);
        this.root = insertTreeNode(this.root, newNode);
    }

    private TreeNode insertTreeNode(TreeNode root, TreeNode newNode) {
        int key = newNode.key;
        if (root == null) {
            root = newNode;
            return root;
        }

        if (key < root.key) {
            root.left = insertTreeNode(root.left, newNode);
        } else if (key > root.key) {
            root.right = insertTreeNode(root.right, newNode);
        }

        return root;
    }

    public int getLevel(int key) {
        int depth = TreeLevel(root, key, 0);
        return depth;
    }

    private int TreeLevel(TreeNode root, int key, int level) {

        if (root == null) {
            return 0;
        }
        if (root.key == key) {
            return level;
        }
        int result = TreeLevel(root.left, key, level + 1);
        if (result != 0) {
            // If found in left subtree , return
            return result;
        }
        result = TreeLevel(root.right, key, level + 1);
    }
}

```

```
        return result;
    }
}
```

```
class Driver {

    public static void main(String args[]) {

        BinarySearchTree tree = new BinarySearchTree();

        tree.insert(44);
        tree.insert(27);
        tree.insert(37);
        tree.insert(60);
        tree.insert(12);
        tree.insert(28);
        tree.insert(62);
        tree.insert(45);

        // tree.inorder();
        // System.out.println("");
        // tree.preorder();
        // System.out.println("");
        // tree.postorder();

        System.out.println(tree.getLevel(12));

    }

}
```